ANEXO DE DOCUMENTACIÓN

Uso de Inteligencia Artificial Generativa

Entrega 2: Diseño e implementación

Proyecto: Proyecto 1 - Sistema de analítica deportiva del Club Deportivo Séneca (CDS)
Propósito del anexo: describir los usos de la IAG como herramienta de apoyo durante la corrección, comprensión, mejora y verificación del código en la Entrega 2.

1. Objetivo y alcance

Durante la Entrega 2, correspondiente al diseño e implementación del sistema, se utilizó Inteligencia Artificial Generativa (IAG) como herramienta complementaria al trabajo desarrollado por los estudiantes. Su uso estuvo orientado a resolver dudas técnicas, depurar errores, apoyar la preparación de módulos de prueba y proponer mejoras sobre código que ya existía en el proyecto.

La IAG se empleó como apoyo y no como sustituto del análisis, las decisiones de diseño ni la responsabilidad del equipo sobre la implementación. Las propuestas debían revisarse en el contexto del sistema, adaptarse cuando fuera necesario y comprobarse antes de incorporarlas al proyecto.

2. Usos de la IAG durante el desarrollo

2.1. Corrección y depuración de código

Se consultó la herramienta para identificar posibles errores en fragmentos de código previamente implementados por los estudiantes, comprender por qué se producían y evaluar alternativas de corrección. Este apoyo incluyó la revisión de condiciones, validaciones, manejo de datos y otros problemas de funcionamiento detectados durante la compilación o ejecución.

2.2. Consultas sobre funcionalidades y conceptos desconocidos

Se realizaron consultas técnicas sobre funcionalidades del lenguaje Java, estructuras de programación y formas de implementar o utilizar recursos que el equipo no conocía con suficiente claridad. Las respuestas se usaron como orientación para comprender las alternativas disponibles y decidir cómo aplicarlas al código del proyecto.

2.3. Elaboración de módulos de prueba

La IAG apoyó la creación de estructuras iniciales para módulos o programas de prueba destinados a ejecutar escenarios concretos y observar el comportamiento de funcionalidades ya implementadas. Estos módulos sirven para facilitar la demostración del sistema y detectar comportamientos inesperados. Los casos y resultados deben corresponder con los requerimientos y reglas de negocio definidos por el equipo.

2.4. Mejora y refactorización de código existente

Se solicitaron sugerencias para mejorar código que ya se encontraba en la base del proyecto, por ejemplo, para aumentar su legibilidad, organizar mejor responsabilidades, reducir repeticiones o facilitar su mantenimiento. Las propuestas se evaluaron procurando conservar el comportamiento esperado y la coherencia entre la implementación, el diseño y la documentación.

3. Proceso de revisión y responsabilidad del equipo

El uso de la herramienta se enmarcó en el siguiente proceso de trabajo:

1. El estudiante identificaba una duda, un error o una parte del código que podía mejorarse y proporcionaba el contexto necesario para formular la consulta.
2. La IAG devolvía una explicación, una posible corrección o una estructura de apoyo. La respuesta se consideraba una sugerencia que podía aceptarse, modificarse o descartarse.

3. Antes de integrar cambios, el equipo debía revisar que la propuesta fuera comprensible y pertinente para la arquitectura y las reglas de negocio del sistema.
4. Los cambios se comprobaban mediante compilación y ejecución, así como mediante los módulos de prueba y los escenarios relevantes para la funcionalidad modificada.
5. La responsabilidad final por el código entregado, sus resultados y la capacidad de explicar su funcionamiento permanecía en los estudiantes.

4. Límites y uso responsable

El enunciado del Proyecto 1 establece que para la Entrega 2 se admite el Nivel 3 de uso de IAG. También indica que no está permitido utilizarla directamente para generar las funcionalidades críticas del sistema, aunque sí contempla su uso para refinar código existente, depurar errores y crear o replicar plantillas o datos de prueba (mock-up). Por ello, el uso descrito en este anexo se circunscribe a esos fines de apoyo y debe reflejar fielmente lo realizado por el equipo.

- No se debe atribuir a la IAG la autoría de decisiones de diseño o funcionalidades que fueron analizadas e implementadas por los estudiantes.
- Las respuestas de la herramienta no se consideran correctas automáticamente: deben contrastarse con el enunciado, el diseño y el comportamiento esperado.
- Los módulos de prueba no reemplazan la validación del sistema; ayudan a ejecutar escenarios y evidenciar resultados.
- La documentación del uso de IAG y la colección de prompts utilizados se incluyen como parte de los anexos del proyecto, según lo solicitado en el enunciado.

5. Registro de usos y verificación

Uso de la IAG	Aplicación en el proyecto	Verificación por el equipo
Corrección de código	Analizar errores de fragmentos previamente escritos y sugerir ajustes.	Revisar la causa del error y comprobar la corrección al compilar y ejecutar.
Consultas técnicas	Aclarar el uso de funcionalidades de Java u otros conceptos requeridos.	Contrastar la explicación con el código y aplicarla solo cuando sea pertinente.
Módulos de prueba	Preparar estructuras de prueba para demostrar comportamientos concretos.	Ejecutar escenarios representativos y comparar el resultado con lo esperado.
Mejora de código existente	Sugerir ajustes de legibilidad, organización y mantenimiento.	Verificar que el cambio no altere reglas de negocio ni contradiga el diseño.

Anexo A. Registro de prompts

El enunciado solicita incluir la colección de prompts utilizados. Como no se proporcionó el historial literal de consultas, los siguientes enunciados son reconstrucciones representativas de los usos descritos, no transcripciones verificadas. Antes de entregar, deben reemplazarse o ajustarse para que coincidan con los prompts realmente utilizados por el equipo.

Propósito	Prompt representativo para ajustar	Resultado esperado / revisión
Corregir código implementado	“Revisa este fragmento de código Java que ya implementamos. Explica el posible error, identifica su causa y propón una corrección mínima sin cambiar el propósito de la funcionalidad	Comprender el error, evaluar el cambio sugerido y compilar/ejecutar el código corregido.
Consultar una funcionalidad desconocida	“Explícame cómo funciona este concepto y muestra un ejemplo pequeño. Indica cómo podría aplicarse a nuestro proyecto y qué aspectos debemos verificar.”	Entender el concepto y decidir si aplica a la implementación existente.
Preparar un módulo de prueba	“A partir de esta funcionalidad ya implementada y de sus reglas, ayúdame a estructurar un programa de prueba que ejecute el escenario [describir escenario], muestre los datos relevantes y permita comparar el resultado con el esperado. No generes la lógica crítica del sistema.”	Revisar que el módulo pruebe una funcionalidad existente, ejecutar el caso y confirmar el resultado.
Mejorar código existente	“Propón mejoras de legibilidad y organización para este código Java ya existente. Explica cada cambio y conserva el comportamiento y las reglas de negocio actuales”	Comparar la propuesta con la versión original, revisar diferencias y probar que el comportamiento se conserve.